
UVM Basics

UVM Basics

Before we can get into discussing the recipes presented in the UVM Cookbook, we have to make sure that we're all talking about the same ingredients. This chapter introduces the UVM concepts that the reader should know in order to understand the recipes presented herein. This section will be incredibly valuable to new UVM users, but experienced UVM users may be able to just straight to the UVM Testbench chapter.

Testbench Basics

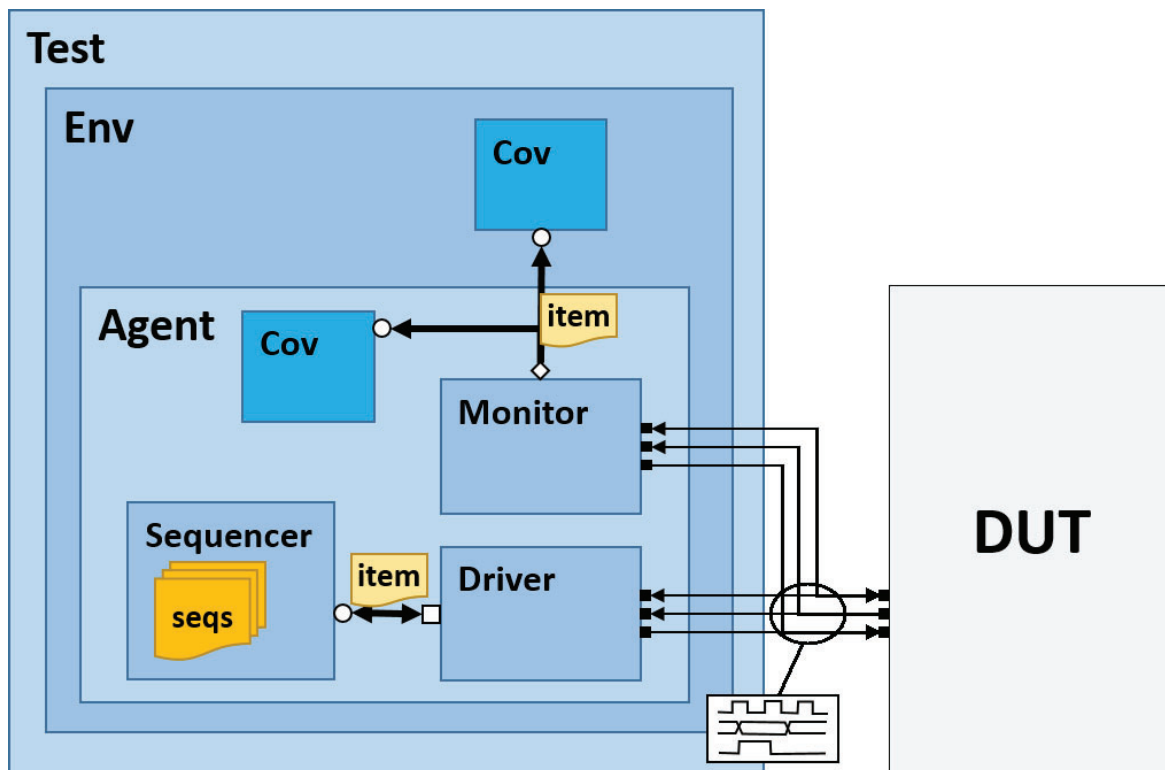
UVM Testbench Basics

The UVM employs a layered, object-oriented approach to testbench development that allows “separation of concerns” among the various team members. Each component in a UVM testbench has a specific purpose and a well-defined interface to the rest of the testbench to enhance productivity and facilitate reuse. When these components are assembled into a testbench, the result is a modular reusable verification environment that allows the test writer to think at the transaction level, focusing on the functionality that must be verified, while the testbench architect focuses on how the test interacts with the Design Under Test (DUT).

The Design Under Test (DUT) is connected to a layer of transactors (drivers, monitors, responders). These transactors communicate with the DUT at the pin level by driving and sampling DUT signals, and with the rest of the UVM testbench by passing transaction objects. They convert data between pins and transactions, i.e. from/to signal to/from transaction level. The testbench layer above the transactor layer consists of components that interact exclusively at the transaction level, such as scoreboards, coverage collectors, stimulus generators, etc. All structural elements in a UVM testbench are extended from the `uvm_component` base class.

The lowest level of a UVM testbench is interface-specific. For each interface, the UVM provides a `uvm_agent` that includes the driver, monitor, stimulus generator (sequencer) and (optionally) a coverage collector. The Agent thus embodies all of the protocol-specific communication with the DUT. The Agent(s) and other design-specific components are encapsulated in a `uvm_env` Environment component which is in turn instantiated and customized by a top-level `uvm_test` component.

The `uvm_sequence_item` – sometimes referred to as a transaction – is a `uvm_object` that contains the data fields necessary to implement the protocol and communicate with the DUT. The `uvm_driver` is responsible for converting the sequence_item(s) into “pin wiggles” on the signal-level interface to send and receive data to/from the DUT. The sequence_items are provided by one or more `uvm_sequence` objects that define stimulus at the transaction level and execute on the agent’s `uvm_sequencer` component. The sequencer is responsible for executing the sequences, arbitrating between them and routing sequence items between the driver and the sequence.



The `uvm_monitor` is responsible for passively observing the pin-level behavior on the DUT interface, converting it into sequence items and providing those sequence items to analysis components in the agent or elsewhere in the testbench such as coverage collectors or scoreboards. UVM Agents also have a *configuration object* that allows the test writer to control aspects of the agent as the testbench is assembled and executed.

By providing a uniform interface to the testbench, a UVM Agent isolates the testbench and the UVM Sequence from details of the interface implementation. A sequence that provides data packets, for example, can be reused with different UVM Agents that may implement AHB, PCI or other protocols. A UVM testbench will typically have one agent per DUT interface.

For a given design, the UVM Agents and other components are encapsulated in a `uvm_env` environment component, which is typically design-specific. Like an agent, an environment typically has a configuration object associated with it that allows the test to control aspects of the environment as well as to control the agents instantiated in the environment. Because environments are themselves UVM components, they can be assembled into a higher-level environment. As block-level designs are assembled into subsystems and systems, the block-level UVM environment associated with the block may be reused as a component in the subsystem-level environment, which can itself be reused in the system-level testbench.

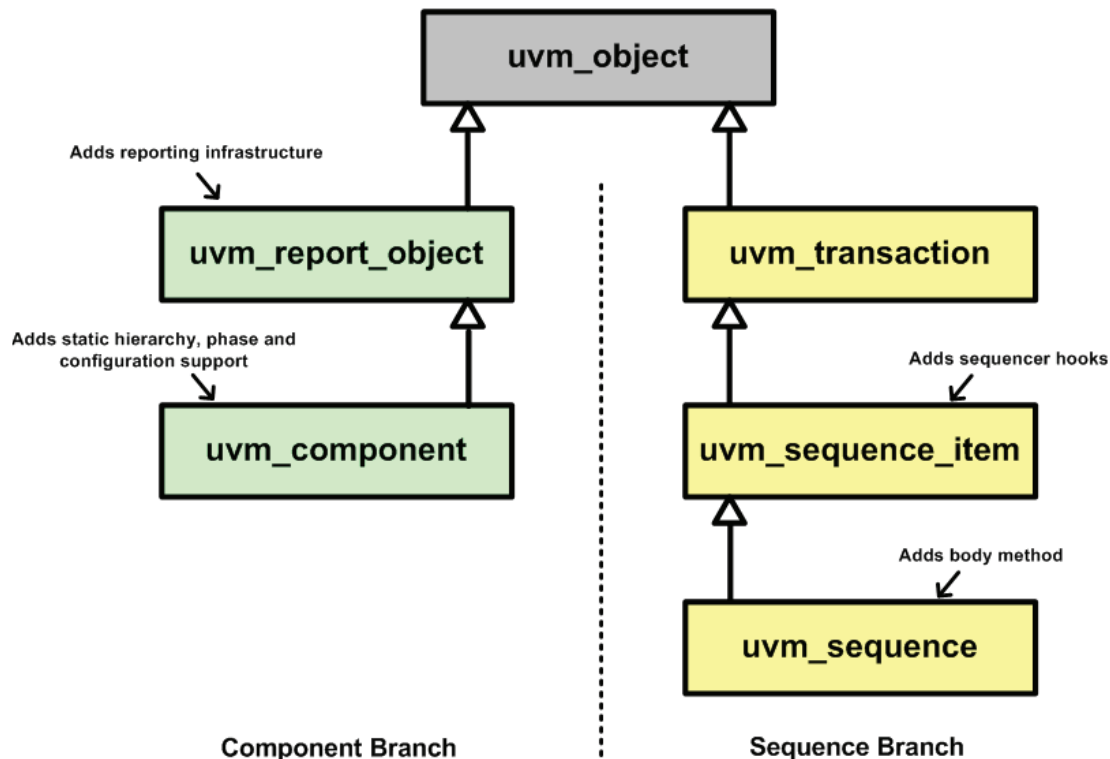
Once the environment has been defined, the `uvm_test` will instantiate, configure and build the environment, including customizing key aspects of the overall testbench, including `..*`choosing variations of components to be used in the environment `..*`choosing UVM Sequences to be run either in the background or as the main portion of the test `..*`defining configuration objects for the environment, sub-environment(s) (if any) and agent(s) in the testbench

The UVM test is started from an initial block in the top-level HVL module by calling `run_test()`.

UVM Components

A UVM testbench is composed of component objects extended from the `uvm_component` base class. When a `uvm_component` derived class object is created, it becomes part of the testbench hierarchy which persists for the duration of the simulation. This contrasts with the sequence branch of the UVM class hierarchy which involves transient objects - objects that are created, used and destroyed (i.e. garbage collected) once dereferenced.

Simplified UVM Inheritance Diagram



The (quasi) static `uvm_component` hierarchy is used by the UVM reporting infrastructure to print the scope of a component issuing a report message, by the configuration process to determine which components can access a configuration object, and by the UVM factory to apply factory overrides. This component hierarchy is represented by a linked list built up incrementally as each component is created. The hierarchical location of each component is determined by the name and parent arguments passed to its create method at the time of construction.

For instance, in the code fragment below, an `apb_agent` component is created within the `spi_env`. Assuming the `spi_env` is instantiated in the top-level test component with the name "m_env," the hierarchical path name of the agent is the concatenation of the `spi_env` component's name, "uvm_test_top.m_env", the "dot" (".") operator, and the name passed as the first argument to the "create()" method, resulting in a hierarchical name for the agent of "uvm_test_top.m_env.m_apb_agent". Any references to the agent would need to use this string name.

```
//
// Hierarchical name example
//
class spi_env extends uvm_env;

// ...
```

```

apb_agent m_apb_agent;

// Declaration of the apb agent handle
// ...

function void build_phase(uvm_phase phase);

    // Create the apb_agent:
    //
    // Name string argument is the same as the handle name
    // The parent argument is 'this' - i.e. the spi_env
    //
    // The spi_env has a hierarchical path string "uvm_test_top.m_env"
    // is concatenated with the name string to arrive at
    // "uvm_test_top.m_env.m_apb_agent" as the
    // hierarchical reference string for the apb_agent

    m_apb_agent = apb_agent::type_id::create("m_apb_agent", this);

    // ...
endfunction: build_phase

// ...
endclass: spi_env

```

The `uvm_component` class inherits from the `uvm_report_object` class which lies at the heart of the UVM Messaging infrastructure. The reporting process uses the component static hierarchy to add the scope of a component to the report message string.

The `uvm_component` base class template has a virtual method for each of the UVM Phases and these are to be implemented by the user as required. A phase level virtual method that is not implemented results in the component effectively not participating in that phase.

Also embedded into the `uvm_component` base class is support for a configuration table to store configuration objects that are relevant for the component's child nodes in the testbench hierarchy. When the `uvm_config_db` API is used, this static hierarchy is employed as part of the path mechanism to control which components may access a given configuration object.

In order to provide flexibility in configuration and to allow the UVM testbench hierarchy to be built in an intelligent way, `uvm_components` are registered with the UVM factory. When a UVM component is created during the build phase, the factory is used to construct the component object. The UVM factory enables a component to be swapped for another of a compatible, derived type using a factory override. This is a useful technique for altering the functionality of a testbench without changing the testbench source code directly, which would require recompilation and hinder reuse. There are a number of coding conventions required for the factory to work and these are outlined in the article on the UVM Factory.

The UVM package contains a number of extensions (i.e. derived classes) of the `uvm_component` base class for common testbench components. Most of these extensions are very "thin", i.e. they are literally just a small extension of the `uvm_component` class to add a new name space. While these are non-critical and in principle a `uvm_component` class could be used instead still, they do help with "self-documentation" as they indicate clearly

the kind of component that is actually represented, such as a driver or monitor. Additionally, there are analysis utilities available that also use these extraneous base classes as clues to help establish a picture of the testbench hierarchy. On the other hand, some of the pre-built `uvm_component` extensions are in fact building blocks providing more profound added value, by instantiate concrete sub-components. The following table summarizes the available UVM component classes directly derived from the `uvm_component` base class:

Class	Description	Contains sub-components?
<code>uvm_driver</code>	Encapsulates sub-components for sequence communication with the <code>uvm_sequencer</code>	No
<code>uvm_sequencer</code>	Encapsulates sub-components for sequence communication with the <code>uvm_driver</code>	No
<code>uvm_subscriber</code>	Encapsulates a <code>uvm_analysis_export</code> and associated virtual <i>write</i> method to implement analysis transaction processing	No
<code>uvm_env</code>	Basis for aggregating verification components around a DUT, or other envs in case of vertical (sub-)system integration	Yes
<code>uvm_test</code>	Basis for a concrete top level test	Yes
<code>uvm_monitor</code>	Basis for a concrete monitor transactor	Yes
<code>uvm_scoreboard</code>	Basis for a concrete scoreboard	Yes
<code>uvm_agent</code>	Basis for concrete agent including a sequencer-driver pair and a monitor	Yes

The UVM Factory

The UVM Factory

The purpose of the UVM factory is to enable an object of one type to be substituted with an object of a derived type without changing the testbench structure or even the testbench code. The mechanism used is referred to as an override, by either instance or type. This functionality is very handy for changing sequence behavior or replacing one version of a component by another. Any two components to be swapped must be polymorphically compatible. This includes the requirement that all the same TLM interface handles and TLM objects must be created by the replacement component. Additionally, in order to take advantage of the factory certain coding conventions must be followed.

Factory Coding Convention 1: Registration

A component or object must contain factory registration code comprised of the following elements:

- A `uvm_component_registry` wrapper, typedefed to `type_id`
- A static function to get the `type_id`
- A function to get the type name

For example:

```
class my_component extends uvm_component;

// Wrapper class around the component class that is used within the factory
typedef uvm_component_registry #(my_component, "my_component") type_id;
```

```

// Used to get the type_id wrapper
static function type_id get_type();
    return type_id::get();
endfunction

// Used to get the type_name as a string
function string get_type_name();
    return "my_component";
endfunction

...
endclass: my_component

```

The registration code has a regular pattern and can be safely generated with one of a set of four factory registration macros:

```

// For a component
class my_component extends uvm_component;

// Component factory registration macro
`uvm_component_utils(my_component)

// For a parameterized component
class my_param_component #(int ADD_WIDTH=20, int DATA_WIDTH=23) extends uvm_component;

typedef my_param_component #(ADD_WIDTH, DATA_WIDTH) this_t;

// Parameterized component factory registration macro
`uvm_component_param_utils(this_t)

// For a class derived from an object (i.e. uvm_object, uvm_transaction,
// uvm_sequence_item, uvm_sequence etc.)

class my_item extends uvm_sequence_item;

    `uvm_object_utils(my_item)

// For a parameterized object class
class my_item #(int ADD_WIDTH=20, int DATA_WIDHT=20) extends uvm_sequence_item;

typedef my_item #(ADD_WIDTH, DATA_WIDTH) this_t

    `uvm_object_param_utils(this_t)

```

Factory Coding Convention 2: Constructor Defaults

The `uvm_component` and `uvm_object` constructors are virtual methods with a prototype template that must be adhered to by users. In order to support deferred construction during the build phase, the factory constructor should contain defaults for the constructor arguments. This allows a factory registered class to be built inside the factory using the defaults and then the class properties can be re-assigned to the arguments passed via the create method of the `uvm_component_registry` wrapper class. The defaults are different for components and objects:

```
// For a component:
class my_component extends uvm_component;

function new(string name = "my_component", uvm_component parent = null);
    super.new(name, parent);
endfunction

// For an object:
class my_item extends uvm_sequence_item;

function new(string name = "my_item");
    super.new(name);
endfunction
```

Factory Coding Convention 3: Component and Object Creation

Testbench components are created during the build phase using the create method of the `uvm_component_registry`. This first constructs the class, then assigns the handle to the class to its declaration handle in the testbench after the name and parent arguments are assigned correctly. For components the build process is top-down, which allows higher level components and configurations to control what actually gets built.

Object classes are created as required, again using the create method. The following code snippet illustrates how this is done:

```
class env extends uvm_env;

my_component m_my_component;
my_param_component #(ADDR_WIDTH(32), DATA_WIDTH(32)) m_my_p_component;

// Constructor & registration macro left out

// Component and parameterized component create examples
function void build_phase( uvm_phase phase );
    m_my_component = my_component::type_id::create("m_my_component", this);
    m_my_p_component = my_param_component #(32, 32)::type_id::create
        ("m_my_p_component", this);
endfunction: build

task run_phase( uvm_phase phase );

my_seq test_seq;
```

```

my_param_seq #(.ADDR_WIDTH(32), .DATA_WIDTH(32)) p_test_seq;

// Object and parameterised object create examples
test_seq = my_seq::type_id::create("test_seq");
p_test_seq = my_param_seq #(32,32)::type_id::create("p_test_seq");
// ...
endtask: run

```

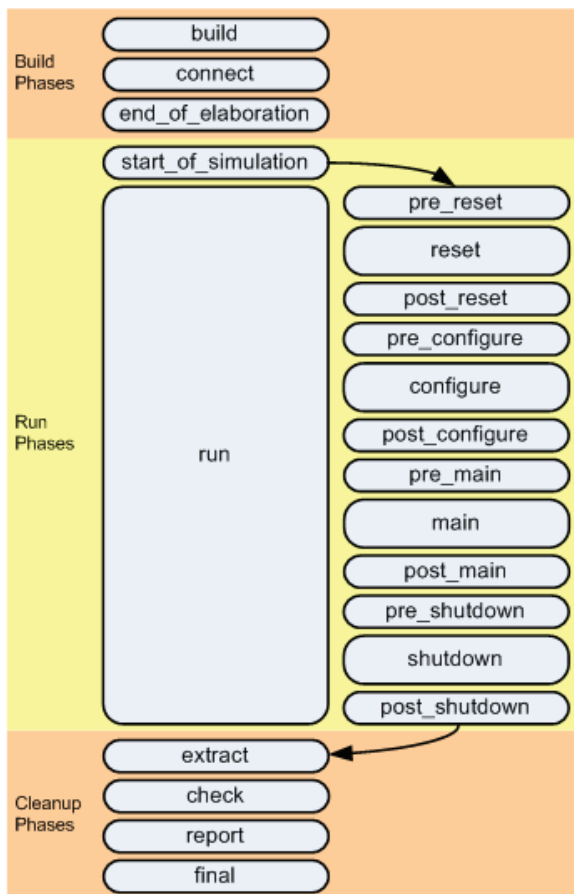
Phasing

The Standard UVM Phases

In order to have a consistent testbench execution flow, the UVM uses phases to order the major steps that take place during simulation. There are three groups of phases, which are executed in the following order:

1. Build phases - where the testbench is configured and constructed
2. Run-time phases - where time is consumed in running the testcase on the testbench
3. Clean up phases - where the results of the testcase are collected and reported

The different phase groups are illustrated in the diagram below. The `uvm_component` base class contains virtual methods which are called by each of the different phase methods and these are populated by the testbench component creator according to which phases the component participates in. Using the defined phases allows verification components to be developed in isolation, but still be interoperable since there is a common understanding of what should happen in each phase.



The UVM Phases

Starting UVM Phase Execution

To start a UVM testbench, the `run_test()` method has to be called from the static part of the testbench. It is usually called from within an initial block in the top level module of the testbench.

Calling `run_test()` constructs the UVM environment root component and then initiates the UVM phasing. The `run_test()` method can be passed a string argument to define the default type name of an `uvm_component` derived class which is used as the root node of the testbench hierarchy. However, the `run_test()` method checks for a command line plusarg called `UVM_TESTNAME` and uses that plusarg string to lookup a factory registered `uvm_component`, overriding any default type name. By convention, the root node will be derived from a `uvm_test` component, but it can be derived from any `uvm_component`. The root node defines the test case to be executed by specifying the configuration of the testbench components and the stimulus to be executed by them.

For instance, in order to specify the test component 'my_test' as the UVM testbench root class, the Questa command line would be:

```
vsim tb_top +UVM_TESTNAME=my_test
```

Phase Descriptions

The following section describes the purpose of each of the different UVM phases

Build Phases

The build phases are executed at the start of the UVM testbench simulation and their overall purpose is to construct, configure and connect the testbench component hierarchy.

All the build phase methods are functions and therefore execute in zero simulation time.

build

Once the UVM testbench root node component is constructed, the build phase starts to execute. It constructs the testbench component hierarchy from the top downwards. The construction of each component is deferred so that each layer in the component hierarchy can be configured by the level above. During the build phase `uvm_components` are indirectly constructed using the UVM factory.

connect

The connect phase is used to make TLM connections between components or to assign handles to testbench resources. It has to occur after the build method has put the testbench component hierarchy in place and works from the bottom of the hierarchy upwards.

end_of_elaboration

The `end_of_elaboration` phase is used to make any final adjustments to the structure, configuration or connectivity of the testbench before simulation starts. Its implementation can assume that the testbench component hierarchy and inter-connectivity is in place. This phase executes bottom up.

Run Time Phases

The testbench stimulus is generated and executed during the run time phases which follow the build phases. After the `start_of_simulation` phase, the UVM executes the run phase and the phases `pre_reset` through to `post_shutdown` in parallel. The run phase was present in the OVM and is preserved to allow OVM components to be easily migrated to the UVM. It is also the phase that transactors will use. The other phases were added to the UVM to give finer run-time phase granularity for tests, scoreboards and other similar components. It is expected that most testbenches will only use reset, configure, main and shutdown and not their pre and post variants

start_of_simulation

The start_of_simulation phase is a function which occurs before the time consuming part of the testbench begins. It is intended to be used for displaying banners; testbench topology; or configuration information. It is called in bottom up order.

run

The run phase occurs after the start_of_simulation phase and is used for the stimulus generation and checking activities of the testbench. The run phase is implemented as a task, and all uvm_component run_phase() tasks are executed in parallel. Transactors such as drivers and monitors will nearly always use this phase.

Parallel Run-Time Phases

NOTE: The following run-time phases execute in-order, in parallel with the run_phase phase. These phases should only be called from the test and the env to start sequences. Drivers, monitors and other components should not implement these phases.

pre_reset

The pre_reset phase starts at the same time as the run phase. Its purpose is to take care of any activity that should occur before reset, such as waiting for a power-good signal to go active. We do not anticipate much use for this phase.

reset

The reset phase is reserved for DUT or interface specific reset behavior. For example, this phase would be used to generate a reset and to put an interface into its default state.

post_reset

The post_reset phase is intended for any activity required immediately following reset. This might include training or rate negotiation behaviour. We do not anticipate much use for this phase.

pre_configure

The pre_configure phase is intended for anything that is required to prepare for the DUT's configuration process after reset is completed, such as waiting for components (e.g. drivers) required for configuration to complete training and/or rate negotiation. It may also be used as a last chance to modify the information described by the test/environment to be uploaded to the DUT. We do not anticipate much use for this phase.

configure

The configure phase is used to program the DUT and any memories in the testbench so that it is ready for the start of the test case. It can also be used to set signals to a state ready for the test case start.

post_configure

The post_configure phase is used to wait for the effects of configuration to propagate through the DUT, or for it to reach a state where it is ready to start the main test stimulus. We do not anticipate much use for this phase.

pre_main

The pre_main phase is used to ensure that all required components are ready to start generating stimulus. We do not anticipate much use for this phase.

main

This is where the stimulus specified by the test case is generated and applied to the DUT. It completes when either all stimulus is exhausted or a timeout occurs. Most data throughput will be handled by sequences started in this phase.

post_main

This phase is used to take care of any finalization of the main phase. We do not anticipate much use for this phase.

pre_shutdown

This phase is a buffer for any DUT stimulus that needs to take place before the shutdown phase. We do not anticipate much use for this phase.

shutdown

The shutdown phase is used to ensure that the effects of the stimulus generated during the main phase have propagated through the DUT and that any resultant data has drained away. It might also be used to execute time consuming sequences that read status registers.

post_shutdown

Perform any final activities before exiting the active simulation phases. At the end of the post_shutdown phase, the UVM testbench execution process starts the clean up phases. We do not anticipate much use for this phase (with the possible exception of some object-to-one code).

Clean Up Phases

The clean up phases are used to extract information from scoreboards and functional coverage monitors to determine whether the test case has passed and/or reached its coverage goals. The clean up phases are implemented as functions and therefore take zero time to execute. They work from the bottom to the top of the component hierarchy.

extract

The extract phase is used to retrieve and process information from scoreboards and functional coverage monitors. This may include the calculation of statistical information used by the report phase. This phase is usually used by analysis components.

check

The check phase is used to check that the DUT behaved correctly and to identify any errors that may have occurred during the execution of the testbench. This phase is usually used by analysis components.

report

The report phase is used to display the results of the simulation or to write the results to file. This phase is usually used by analysis components.

final

The final phase is used to complete any other outstanding actions that the testbench has not already completed.

UVM Driver

Overview

The UVM driver is responsible for communicating at the transaction level with the sequence via TLM communication with the sequencer and converting between the `sequence_item` on the transaction side and pin-level activity in communicating with the DUT via a virtual interface. As the name implies, drivers typically get a `sequence_item` and use that information to drive signals to the DUT and may, in certain applications, also receive a pin-level response from the DUT and convert it back into a `sequence_item` for the sequence to complete the transaction. A driver may also function as a "responder" (i.e. in "slave mode") in which the driver reacts to pin-level activity in the interface to communicate with a sequence that then sends a response transaction back to the driver to complete the protocol transaction. When its agent is configured to be in passive mode, the driver is, by definition, not instantiated in the agent.

Anatomy of a UVM Driver

A user-defined driver component is a proxy class derived from a *uvm_driver* base class and contains a BFM which is a SystemVerilog interface. The *uvm_driver* base class provides a `seq_item_port` that gets connected by the agent to the `seq_item_export` of the agent's *uvm_sequencer*. Usually, responses are passed back to the sequence through the `seq_item_port` as well, but certain applications may require that responses be sent back to the sequencer via the *rsp_port* of the driver. For a detailed discussion of connecting a driver to a sequencer, please see Sequencer-Driver Connections.

UVM Driver API

The *uvm_driver* is designed ultimately to interact with a *uvm_sequence* running on the connected *uvm_sequencer*. The details of this API are described here.

UVM Driver Use Models

Stimulus generation in the UVM relies on a coupling between sequences and drivers. A sequence can only be written when the characteristics of a driver are known, otherwise there is a potential for the sequence or the driver to get into a deadlock waiting for the other to provide an item. This problem can be mitigated for reuse by providing a set of base utility sequences which can be used with the driver and by documenting the behavior of the driver.

There are a large number of potential stimulus generation use models for the sequence driver combination, most of which are discussed here.

UVM Monitor

Overview

The first task of the analysis portion of the testbench is to monitor activity on the DUT. A Monitor, like a Driver, is a constituent of an agent. A monitor component is similar to a driver component in that they both perform a translation between actual signal activity and an abstract representation of that activity. The key difference between a Monitor and a Driver is that a Monitor is always passive. It does not drive any signals on the interface. When an agent is placed in passive mode, the Monitor continues to execute.

A Monitor communicates with DUT signals through a virtual interface, and contains code that recognizes protocol patterns in the signal activity. Once a protocol pattern is recognized, a Monitor builds an abstract transaction model representing that activity, and broadcasts the transaction to any interested components.

Construction

Monitors are composed of a proxy class which should extend from `uvm_monitor` and a BFM which is a SystemVerilog interface. The proxy should have one analysis port and a virtual interface handle that points to a BFM interface.

```
class wb_bus_monitor extends uvm_monitor;
  `uvm_component_utils(wb_bus_monitor)

  uvm_analysis_port #(wb_txn) wb_mon_ap;
  virtual wb_bus_monitor_bfm m_bfm; //BFM handle
  wb_config m_config;

  // Standard component constructor
  function new(string name, uvm_component parent);
    super.new(name,parent);
  endfunction

  function void build_phase( uvm_phase phase );
    wb_mon_ap = new("wb_mon_ap", this);
    m_config = wb_config::get_config(this); // get config object
    m_bfm = m_config.WB_mon_bfm; // set local virtual if property
    m_bfm.proxy = this; //Set BFM proxy handle
  endfunction

  task run_phase(uvm_phase phase);
    m_bfm.run(); //Don't start the BFM until we get to the run_phase
  endtask

  function void notify_transaction(wb_txn item); //Used by BFM to
return transactions
    wb_mon_ap.write(item);
  endfunction : notify_transaction

endclass
```

```

interface wb_bus_monitor_bfm (wishbone_bus_syscon_if wb_bus_if);

    import wishbone_pkg::*;

    //-----
    // Data Members
    //-----
    wb_bus_monitor proxy;

    //-----
    // Methods
    //-----

    task run();
        wb_txn txn;

        forever @ (posedge wb_bus_if.clk)
            //Capture protocol pin activity into txn
            proxy.notify_transaction(txn);
        end
    endtask

endinterface

```

Passive Monitoring

As in a scientific experiment the act of observing should not affect the activity observed, monitor components should be passive. They should not inject any activity into the DUT signals. Practically speaking, this means that monitor code should be completely read-only when interacting with DUT signals. Additionally, the BFM will not be observing data until instructed to do so by the proxy to ensure that the BFM is aligned with UVM phasing.

Recognizing Protocols

A monitor must have knowledge of a protocol in order to detect recognizable patterns in signal activity.

Detection can be done by writing protocol-specific state machine code in the monitor BFM's `run()` task. This code waits for a pattern of key signal activity by observing through the pin interface handle.

Building Transaction Objects

Once the pattern is recognized, monitors build one or possibly more transactions that abstractly represents the signal activity. This can be done by calling a function and passing in the transaction-specific attributes (e.g. data value and address value) as arguments to the function, or by setting transaction attributes on an existing transaction as they are detected.

Copy-on-Write Policy

Since objects in SystemVerilog are handle-based, when a Monitor writes a transaction handle out of its analysis port, only the handle gets copied and broadcast to subscribers. This write operation happens each time the Monitor runs through its ongoing loop of protocol recognition in the `run()` task. To prevent overwriting the same object memory in the next iteration of the loop, the handle that is broadcast should point to a separate copy of the transaction object that the Monitor created.

This can be accomplished in two ways:

- Create a new transaction object in each iteration of (i.e. inside) the loop
- Reuse the same transaction object in each iteration of the loop, but clone the object immediately prior to calling write() and broadcast the handle of the clone.

Broadcasting the Transaction

Once a new or cloned transaction has been built it should be broadcast to all interested observers by writing to an analysis port.

Example Monitor

```
//Full run task from monitor BFM
task run();
wb_txn txn;

forever @ (posedge wb_bus_if.clk)
    if(wb_bus_if.s_cyc) begin
        // Is there a valid wb cycle?
        txn = wb_txn::type_id::create("txn");
        // create a new wb_txn
        txn.adr = wb_bus_if.s_addr;
        // get address
        txn.count = 1;
        // set count to one read or write
        if(wb_bus_if.s_we) begin
            // is it a write?
            txn.data[0] = wb_bus_if.s_wdata;
            // get data
            txn.txn_type = WRITE;
            // set op type
            while (!(wb_bus_if.s_ack[0] | wb_bus_if.s_ack[1] | wb_bus_if.s_ack[2]))
                @ (posedge wb_bus_if.clk);
            // wait for cycle to end
        end
    else begin
        txn.txn_type = READ;
        // set op type
        case (1)
            //Nope its a read, get data from correct slave
            wb_bus_if.s_stb[0]: begin
                while (!(wb_bus_if.s_ack[0])) @ (posedge wb_bus_if.clk);
            //wait for ack
            txn.data[0] = wb_bus_if.s_rdata[0];
            // get data
        end
```

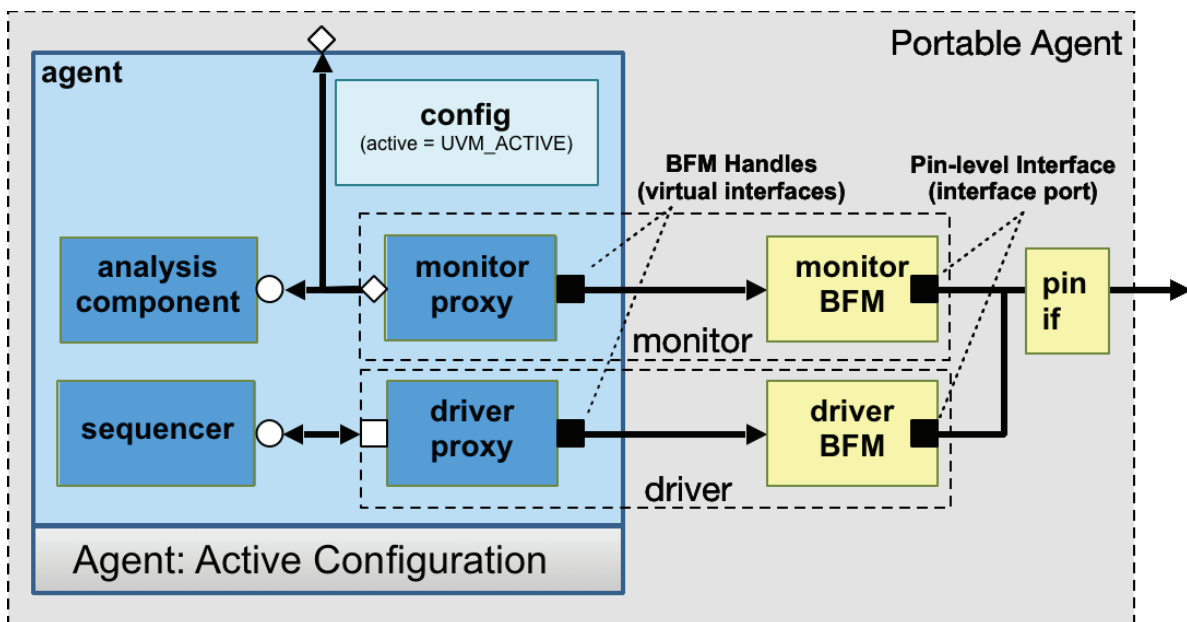
```

wb_bus_if.s_stb[1]: begin
    while (!(wb_bus_if.s_ack[1])) @ (posedge wb_bus_if.clk);
    // wait for ack
    txn.data[0] = wb_bus_if.s_rdata[1];
    // get data
end
endcase
end
// else: !if(wb_bus_if.s_we)
proxy.notify_transaction(txn);
end
endtask

```

UVM Agent

A UVM agent is a verification component "kit" for a given logical interface such as APB or USB. The agent includes a SystemVerilog interface encapsulating the corresponding set of interface signals, two SystemVerilog interfaces representing the monitor and driver BFM, and a SystemVerilog package including the various classes that make up the overall agent class component. The agent class itself is a container class for a sequencer, a driver proxy and a monitor proxy, plus any other verification components deemed relevant such as a functional coverage collector or a scoreboard. Proxies are simply class objects providing the same API as "normal" class objects. The driver proxy and monitor proxy communicate with the rest of a UVM testbench in the usual way, while also accessing respectively the driver and monitor BFM interface via a virtual interface handle. A complete monitor is thus composed of the monitor proxy and monitor BFM working together as a pair, and the same is true for a driver^[1]. The agent also has an analysis port that is connected to the analysis port of the monitor, enabling a user to connect external analysis components to the agent without needing to know its internal structure. The agent is the lowest level hierarchical block in a testbench and its exact structure depends on its configuration, which can vary from one test to another per the agent configuration object. The classes and interfaces together constitute a portable, or reusable Agent.



Let's examine how an APB agent is composed, configured, built, and connected. The APB agent's pin interface, `apb_if`, is coded in the file `apb_if.sv`. The monitor BFM interface named `apb_monitor_bfm` has an `apb_if` port. The

driver BFM named `apb_driver_bfm` also has an `apb_if` port. The BFMs define tasks and functions to interact with the signals in the `apb_if` pin interface. The driver and monitor proxies do not directly access the pins, which is to be kept local to the BFMs. The file `apb_agent_pkg.sv` contains the SystemVerilog package with the various class files for the APB agent. Any component using files from this package, like an `env`, imports this package.

```
package apb_agent_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"
`include "config_macro.svh"

`include "apb_seq_item.svh"
`include "apb_agent_config.svh"
`include "apb_driver.svh"
`include "apb_coverage_monitor.svh"
`include "apb_monitor.svh"
typedef uvm_sequencer#(apb_seq_item) apb_sequencer;
`include "apb_agent.svh"

//Reg Adapter for UVM Register Model
`include "reg2apb_adapter.svh"

// Utility Sequences
`include "apb_seq.svh"
`include "apb_read_seq.svh"
`include "apb_write_seq.svh"

endpackage: apb_agent_pkg
```

Note that the `apb_sequencer` type is actually a typedef that simply parameterizes the `uvm_sequencer` base component with the `sequence_item` type being used.

The Agent Configuration Object

The agent has a configuration object which defines:

- The topology of the agent's sub-components (determines what gets constructed)
- The handles for the BFM virtual interfaces used by the driver and monitor proxies
- The behavior of the agent

By convention, UVM agent configuration classes have a data member of type `uvm_active_passive_enum` that defines whether the agent is active (`UVM_ACTIVE`) with the sequencer and driver proxy constructed, or passive (`UVM_PASSIVE`) with neither the driver proxy nor sequencer constructed. This parameter is called `active` and by default set to `UVM_ACTIVE`.

Whether other sub-components are built or not is controlled by additional configuration data members which should have descriptive names. For instance, if there is a functional coverage collector, there ought to be a bit that controls whether this coverage collector is built or not, suitably named `has_functional_coverage`.

The configuration object contains handles for the BFM virtual interfaces used by the driver proxy and the monitor proxy. The configuration object is constructed and configured in the test and it is at this top level where the virtual interface handles are assigned to the virtual interfaces passed in from the testbench module.

The agent configuration object may also contain other data members to control how the agent is configured or behaves. For instance, the configuration object for the APB agent has data members to set up the memory map and determine the APB PSEL lines that are activated with associated address map.

The configuration class should have all configuration data members default to common values.

The following code example shows the configuration object for the APB agent.

```
//
// Class Description:
//
//
class apb_agent_config extends uvm_object;

// UVM Factory Registration Macro
//
`uvm_object_utils(apb_agent_config)

// BFM Virtual Interfaces
virtual apb_monitor_bfm mon_bfm;
virtual apb_driver_bfm drv_bfm;

//-----
// Data Members
//-----
// Is the agent active or passive
uvm_active_passive_enum active = UVM_ACTIVE;
// Include the APB functional coverage collector
bit has_functional_coverage = 0;
// Include the APB RAM based scoreboard
bit has_scoreboard = 0;
//
// Address decode for the select lines:
// Address decode for the select lines:
int no_select_lines = 1;
int apb_index = 0;
// Which PSEL is the monitor connected to
logic[31:0] start_address[15:0];
logic[31:0] range[15:0];

//-----
// Methods
//-----

// Standard UVM Methods:
extern function new(string name = "apb_agent_config");

endclass: apb_agent_config

function apb_agent_config::new(string name = "apb_agent_config");
```


The Agent Connect Phase

Once the agent's sub-components have been constructed, they must be connected. The usual agent connections required are:

- Monitor's analysis port to agent's analysis port ¹
- Sequencer's seq_item_pull_export to driver's seq_item_pull_port (for active agent only)
- Analysis sub-component's analysis export to monitor's analysis port (where analysis sub-components exist)
- Monitor/driver's proxy virtual interface to configuration object's monitor/driver BFM virtual interface ²

Notes:

1. The agent's analysis port handle can be assigned a pointer from the monitor's analysis port. This saves having to construct a separate analysis port object in the agent.
2. Assigning the driver proxy and monitor proxy virtual interfaces in the agent removes the need for these sub-components to have the overhead of a configuration table lookup.

The following code for the APB agent illustrates how the configuration object determines what happens during the build and connect phases:

```
//
// Class Description:
//
//
class apb_agent extends uvm_component;

// UVM Factory Registration Macro
//
`uvm_component_utils(apb_agent)

//-----
// Data Members
//-----
apb_agent_config m_cfg;
//-----
// Component Members
//-----
uvm_analysis_port #(apb_seq_item) ap;
apb_monitor m_monitor;
apb_sequencer m_sequencer;
apb_driver m_driver;
apb_coverage_monitor m_fcov_monitor;
//-----
// Methods
//-----

// Standard UVM Methods:
extern function new(string name = "apb_agent", uvm_component parent = null);
extern function void build_phase(uvm_phase phase);
extern function void connect_phase(uvm_phase phase);
```

```

endclass: apb_agent

function apb_agent::new(string name = "apb_agent", uvm_component parent
= null);
    super.new(name, parent);
endfunction

function void apb_agent::build_phase( uvm_phase phase );
    if (m_cfg == null)
        if( !uvm_config_db #( apb_agent_config )::get(this, "",
"apb_agent_config",m_cfg) ) `uvm_fatal(...)
            // Monitor is always present
        m_monitor = apb_monitor::type_id::create("m_monitor", this);
        // Only build the driver and sequencer if active
        if(m_cfg.active == UVM_ACTIVE) begin
            m_driver = apb_driver::type_id::create("m_driver", this);
            m_sequencer = apb_sequencer::type_id::create("m_sequencer", this);
        end
        if(m_cfg.has_functional_coverage) begin
            m_fcov_monitor =
apb_coverage_monitor::type_id::create("m_fcov_monitor", this);
        end
endfunction: build_phase

function void apb_agent::connect_phase(uvm_phase phase);
    ap = m_monitor.ap;
    // Only connect the driver and the sequencer if active
    if(m_cfg.active == UVM_ACTIVE) begin
        m_driver.seq_item_port.connect(m_sequencer.seq_item_export);
    end
    if(m_cfg.has_functional_coverage) begin
        m_monitor.ap.connect(m_fcov_monitor.analysis_export);
    end
endfunction: connect_phase

```

The build process for the APB agent can be followed in the block level testbench example:

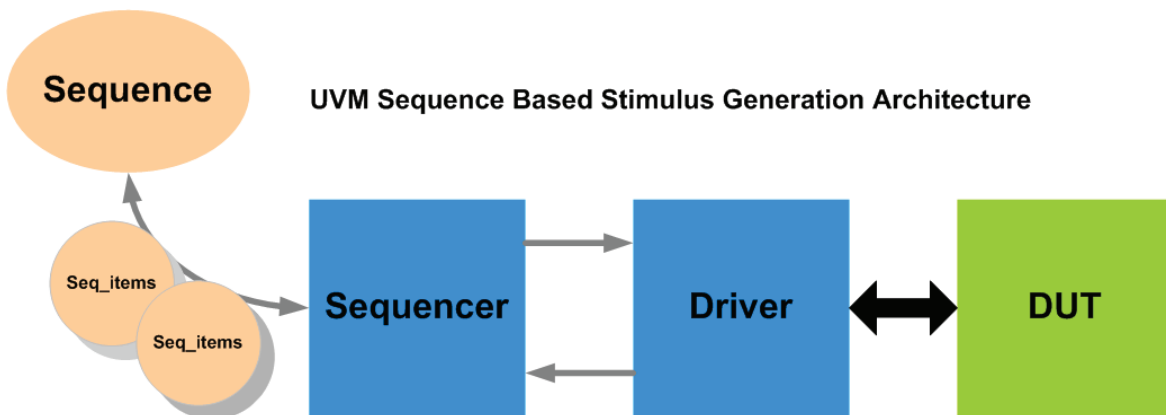
References

[1] <https://verificationacademy.com/patterns-library/implementation-patterns/environment-patterns/bfm-proxy-pair-pattern>

UVM Sequences

Sequence Overview

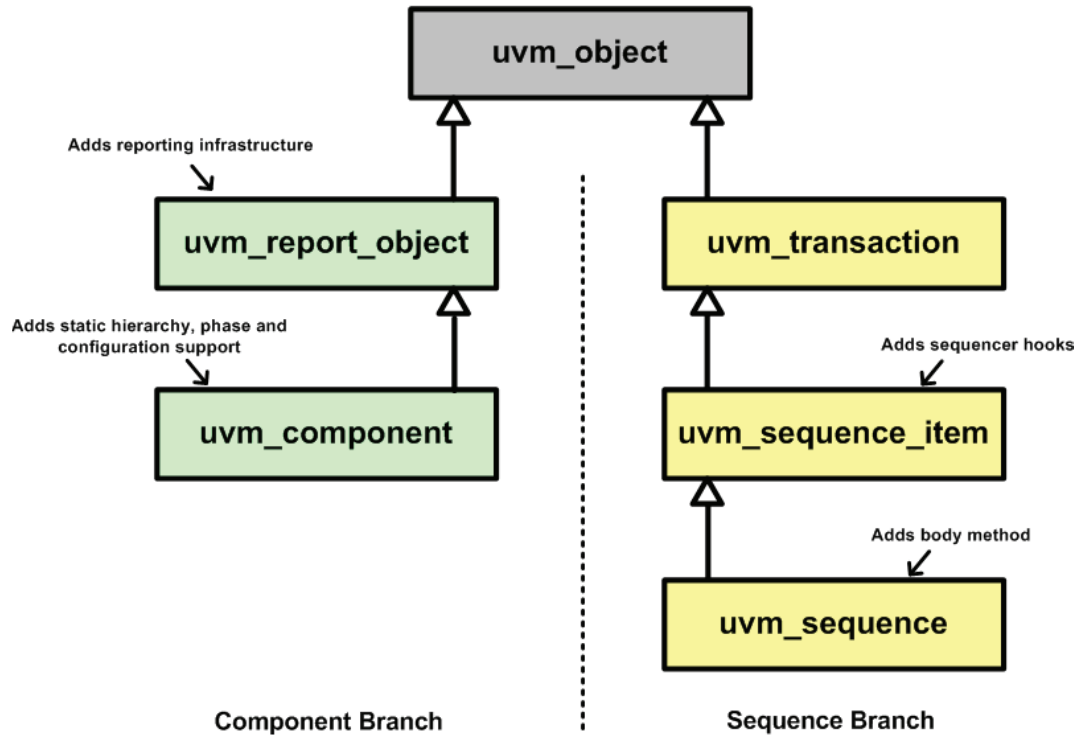
In testbenches written in traditional HDLs like Verilog and VHDL, stimulus is generated by layers of sub-routine calls which either execute time consuming methods (i.e. Verilog tasks or VHDL processes or procedures) or call non-time consuming methods (i.e. functions) to manipulate or generate data. Test cases implemented in these HDLs rely on being able to call sub-routines which exist as elaborated code at the beginning of the simulation. There are several disadvantages with this approach - it is hard to support constrained random stimulus; test cases tend to be 'hard-wired' and quite difficult to change and running a new test case usually requires recompiling the testbench. Sequences bring an Object Orientated approach to stimulus generation that is very flexible and provides new options for generating stimulus.



A sequence is an example of what software engineers call a 'functor', in other words it is an object that is used as a method. An UVM sequence contains a task called body. When a sequence is used, it is created, then the body method is executed, and then the sequence can be discarded. Unlike an `uvm_component`, a sequence has a limited simulation life-time and can therefore be described as a transient object. The sequence body method can be used to create and execute other sequences, or it can be used to generate `sequence_item` objects which are sent to a driver component, via a sequencer component, for conversion into pin-level activity or it can be used to do some combination of the two. The `sequence_item` objects are also transient objects, and they contain the information that a driver needs in order to carry out a pin level interaction with a DUT. When a response is generated by the DUT, then a `sequence_item` is used by the driver to pass the response information back to the originating sequence, again via the sequencer. Creating and executing other sequences is effectively the same as being able to call conventional sub-routines, so complex functions can be built up by chaining together simple sequences.

In terms of class inheritance, the `uvm_sequence` inherits from the `uvm_sequence_item` which inherits from the `uvm_object`. Both base classes are known as objects rather than components. The UVM testbench component hierarchy is built from `uvm_components` which have different properties, which are mainly to do with them being tied into a static component hierarchy as they are built and that component hierarchy stays in place for the life-time of the simulation.

Simplified UVM Inheritance Diagram



Sequences are the primary means of generating stimulus in the UVM. The fact that sequences and sequence_items are objects means that they can be easily randomized to generate interesting stimulus. Their object orientated nature also means that they can be manipulated in the same way as any other object. The UVM architecture also keeps these classes separate from the testbench component hierarchy, the advantage being that it is easy to define a new test case by calling and executing different combinations of sequences from a library package without being locked to the methods available within the scope of the components. The disadvantage is that sequences are not able to directly access testbench resources such as configuration information, or handles to register models, which are available in the component hierarchy. Sequences access testbench resources using a sequencer as the key into the component hierarchy.

In the UVM sequence architecture, sequences are responsible for the stimulus generation flow and send sequence_items to a driver via a sequencer component. The driver is responsible for converting the information contained within sequence_items into pin level activity. The sequencer is an intermediate component which implements communication channels and arbitration mechanisms to facilitate interactions between sequences and drivers. The flow of data objects is bidirectional, request items will typically be routed from the sequence to the driver and response items will be returned to the sequence from the driver. The sequencer end of the communication interface is connected to the driver end together during the connect phase.

UVM Sequence Items

The UVM stimulus generation process is based on sequences controlling the behavior of drivers by generating sequence_items and sending them to the driver via a sequencer. The framework of the stimulus generation flow is built around the sequence structure for control, but the generation data flow uses sequence_items as the data objects.

As sequence_items are the foundation on which sequences are built, some care needs to be taken with their design. Sequence_item content is determined by the information that the driver needs in order to execute a pin level transaction; ease of generation of new data object content, usually by supporting constrained random generation; and other factors such analysis hooks.

Data Property Members

The content of the sequence_item is closely related to the needs of the driver. The driver relies on the content of the sequence_items it receives to determine which type of pin level transaction to execute. The sequence items property members will consist of data fields that represent the following types of information:

- Control - i.e. What type of transfer, what size
- Payload - i.e. The main data content of the transfer
- Configuration - i.e. Setting up a new mode of operation, error behavior etc
- Analysis - i.e. Convenience fields which aid analysis - time stamps, rolling checksums etc

Randomization Considerations

Sequence_items are randomized within sequences to generate traffic data objects. Therefore, stimulus data properties should generally be declared as rand, and the sequence_item should contain any constraints required to ensure that the values generated by default are legal, or are within sensible bounds. In a sequence, sequence_items are often randomized using in-line constraints which extend these base level constraints.

As sequence_items are used for both request and response traffic and a good convention to follow is that request properties should be rand, and that response properties should not be rand. This optimizes the randomization process and also ensures that any collected response information is not corrupted by any randomization that might take place.

For example consider the following bus protocol sequence_item:

```
class bus_seq_item extends uvm_sequence_item;

  // Request data properties are rand
  rand logic[31:0] addr;
  rand logic[31:0] write_data;
  rand bit read_not_write;
  rand int delay;

  // Response data properties are NOT rand
  bit error;
  logic[31:0] read_data;

  `uvm_object_utils(bus_seq_item)

function new(string name = "bus_seq_item");
  super.new(name);
endfunction
```



```
// Delay between bus cycles is in a sensible range
constraint at_least_1 { delay inside {[1:20]};}

// 32 bit aligned transfers
constraint align_32 {addr[1:0] == 0;}

// etc
endclass: bus_seq_item
```

Sequence Item Methods

The `uvm_sequence_item` inherits from the `uvm_object` via the `uvm_transaction` class. The `uvm_object` has a number of virtual methods which are used to implement common data object functions (copy, clone, compare, print, transaction recording) and these should be implemented to make the `sequence_item` more general purpose.

A `sequence_item` is often used in analysis traffic and it may be useful to add utility functions which aid functional coverage or analysis.

UVM Configuration Database (`uvm_config_db`)

The `uvm_config_db` class is the recommended way to access the resource database. A resource is any piece of information that is shared between two or more components or objects. Use `uvm_config_db::set` to put information into the database and use `uvm_config_db::get` to retrieve information from the database. The `uvm_config_db` class is a type-parameterized class, and consequently the database behaves as if it were partitioned into many type-specific "mini databases." There are no limitations on the type parameter, which can be a class, a `uvm_object`, a built-in type like a bit, byte, or a virtual interface, etc.

There are two typical uses of the `uvm_config_db`. The first is to pass virtual interfaces from the HDL/DUT domain to the test, and the second is to pass configuration objects down through the testbench hierarchy.

The set method

The full signature of the set method is `void uvm_config_db #( type T = int )::set( uvm_component cntxt , string inst_name , string field_name , T value );`

- **T** is the type of the resource, or element, being added - usually a virtual interface or a configuration object.
- **cntxt** and **inst_name** together form a scope that is used to locate the resource within the database; it is formed by appending the instance name to the full hierarchical name of the context, i.e. `{cntxt.get_full_name(),".",inst_name}`.
- **field_name** is the name given to the resource.
- **value** is the actual value or reference that is put into the database.

An example of putting virtual interfaces into the UVM configuration database is as follows:

```
interface ahb_if data_port_if( clk , reset );
interface ahb_if control_port_if( clk , reset );
...
uvm_config_db #( virtual ahb_if )::set( null , "uvm_test_top" ,
"data_port" , data_port_if );
uvm_config_db #( virtual ahb_if )::set( null , "uvm_test_top" ,
```

```
"control_port" , control_port_if );
```

This code puts two AHB interfaces into the configuration database at the hierarchical location "uvm_test_top", which is the default location for the top level test component created by `run_test()`. The two different interfaces are put into the database using two distinct names, "data_port" and "control_port".

Notes:

- Use **"uvm_test_top"** because it is more precise and hence more efficient than **"*"**; it works in general except when your top level test component does something other than `super.new( name , parent )` in its constructor, in which case the instance name must be adjusted accordingly.
- Use **"null"** in the first argument as this code is in a top level module rather than a `uvm_component`.
- Use the parameterized `uvm_config_db::set()` method instead of the `set_config_[int,string,object]()` methods which are deprecated in UVM1.2. See UVM1.2 Summary for more details.

An example of configuring agents inside an env is as follows:

```
class env extends uvm_env;

    ahb_agent_config m_ahb_agent_config;

    function void build_phase( uvm_phase phase );
        ...
        m_ahb_agent = ahb_agent::type_id::create("m_ahb_agent" , this );
        ...
        uvm_config_db #( ahb_agent_config )::set( this , "m_ahb_agent*" ,
"ahb_agent_config" , m_ahb_agent_config );
        ...
    endfunction
endclass
```

This code sets the configuration for the AHB agent and all its child components. Two things to note:

- Use **"this"** as the first argument to ensure that only *this* agent's configuration is set, and not of any other `ahb_agent` in the component hierarchy.
- Use **"m_ahb_agent*"** to ensure that both the agent and its children are in the look-up scope. Without the **"*"** only the agent itself would be, and its driver, sequencer and monitor sub-components would be unable to access the configuration.

The get method

The full signature of the get method is **bit uvm_config_db #(type T = int)::get(uvm_component cntxt , string inst_name , string field_name , ref T value);**

- **T** is the type of the resource, or element, being retrieved - usually a virtual interface or a configuration object.
- **cntxt** and **inst_name** together form a scope that is used to locate the resource within the database; it is formed by appending the instance name to the full hierarchical name of the context, i.e. `{cntxt.get_full_name(),".",inst_name}`.
- **field_name** is the name given to the resource.
- **value** holds the actual value or reference that is retrieved from the database; the `get()` call **returns** 1 if that retrieval succeeds, or else 0 indicating that no resource of this type and with this context and name exists in the database.

An example of getting virtual interfaces from the configuration database is as follows:

```

class test extends uvm_test;
...
function void build_phase( uvm_phase phase );
...
if( !uvm_config_db #( virtual ahb_if )::get( this , "" , "data_port" ,
m_cfg.m_data_port_config.m_ahb_if ) ) begin
    `uvm_error("Config Error", "uvm_config_db #( virtual ahb_if )::get
cannot find resource data_port" ) )
end
...
endfunction
...
endclass

```

The code attempts to get the virtual interface for the AHB dataport and assign it into the correct agent's configuration object. A meaningful error message is provided when the database lookup fails.

An example of retrieving configuration for a transactor is as follows:

```

class ahb_monitor extends uvm_monitor;
    ahb_agent_config m_cfg;

    function void build_phase( uvm_phase phase );
    ...
    if( !uvm_config_db #( ahb_agent_config )::get( this , "" ,
"ahb_agent_config" , m_cfg ) ) begin
        `uvm_error("Config Error" , "uvm_config_db #( ahb_agent_config
)::get cannot find resource ahb_agent_config" )
    end
    ...
    endfunction
endclass

```

Again, a few notes are in order:

- Use **"this"** as the context argument.
- Use **""** as the instance name.
- Use the return value of the `get()` call to check whether it fails and a useful error message should be given.
- Use the parameterized `uvm_config_db::get()` method instead of the `get_config_[int,string,object]()` methods which are deprecated in UVM1.2. See UVM1.2 Summary for more details.

Precedence Rules

Two sets of precedence rules apply to `uvm_config_db`. Firstly, during the build phase a `set()` call in a context higher up the component hierarchy takes precedence over a `set()` call occurring lower down the hierarchy. Secondly, in case of identical contexts or after the build phase, the last `set()` call takes precedence over the earlier one. For more details on these rules, please consult the UVM reference manual or look directly at the verbosity messages from the UVM code.

Using Packages

A package is a SystemVerilog language construct that enables related declarations and definitions to be grouped together in a package namespace. A package might contain type definitions, constant declarations, functions and class templates. To use a package within a scope, it must be imported, after which its contents can be referenced.

A SystemVerilog package is a useful means to organize code and to make sure that references to types, classes etc. are consistent. The UVM base class library is contained within one package called the "uvm_pkg". Packages should be used for developing UVM testbenches to collect and organize the various class definitions developed to implement agents, envs, sequence libraries, test libraries and so forth.

UVM Package Coding Guidelines

Naming a package and package file

A package should be named with a `_pkg` suffix. The name of the file containing the package should reflect the name of the package and have a `.sv` extension. As an example, the file `spi_env_pkg.sv` would contain the package `spi_env_pkg`.

Justification: The `.sv` extension is a convention that denotes that the package file is a stand-alone compilation unit. The `_pkg` suffix denotes that the file contains a package. Both of these conventions are useful to humans and machine parsing scripts.

Including classes in a package

Class templates that are declared within the scope of a package should be separated out into individual files with a `.svh` extension. These files should be ``included` in the package in the order in which they need to be compiled. The package file is the only place where ``includes` should be used, i.e. there should be no further ``include` statements inside the ``included` files themselves.

Justification: Declaring classes in separate individual files makes them easier to maintain, and also conveys the package content more clearly.

Importing packages and package elements

The contents of a package may reference contents of other packages as enabled by package imports. Such external package imports should be declared at the top of the package as the first statements of the package "body". Individual files like class templates that may be included should not do separate imports.

Justification: Localizing all package imports in one place makes the package dependencies clear. Placing package imports in other parts of the package or inside ``included` files is less visible and more likely to cause ordering issues and potential type clashes.

Organizing package files into a directory

All files included in a given package should be put together in a single directory. This is particularly important for agents where the agent directory structure needs to be a complete stand-alone package.

Justification: A single include directory for package files facilitates compilation flow set-up, and also aids reuse since all the files for a package can be gathered together easily.

Below is an example of a package file for a UVM env. This env contains two agents (SPI and APB) and a register model, which are imported as sub-packages. The class templates relevant to the env are `included:

```
// Note that this code is contained in a file called spi_env_pkg.sv
//
// In Questa it would be compiled using:
// vlog +incdir+$UVM_HOME/src+<path_to_spi_env> <path_to_spi_env>/spi_env_pkg.sv
//

//
// Package Description:
//
package spi_env_pkg;

// Standard UVM import & include:
import uvm_pkg::*;
`include "uvm_macros.svh"

// Any further package imports:
import apb_agent_pkg::*;
import spi_agent_pkg::*;
import spi_register_pkg::*;

// Includes:
`include "spi_env_config.svh"
`include "spi_virtual_sequencer.svh"
`include "spi_env.svh"

endpackage: spi_env_pkg
```

Package Scopes

Users are often confused by the fact that the SystemVerilog package defines a scope. This means that everything declared within a package, along with the elements of other packages imported into the package are visible only within the scope of that package. If a package is imported into another scope (i.e. another package or a module), only the elements of the imported package are visible and not the elements of any packages imported by the imported package itself. Hence, if elements of these other packages are required in the new scope, they need to be imported separately.

```
//  
// Package Scope Example  
// -----  
//  
package spi_test_pkg;  
  
// The UVM package has to be imported, even though it is imported in the spi_env package.  
// This is because the import of the uvm_pkg is only visible within the current scope  
import uvm_pkg::*;  
// The same is true of the `include of the uvm_macros  
`include "uvm_macros.svh"  
  
// Import of uvm_pkg inside the spi_env package is not  
// visible within the current spi_test_pkg scope  
import spi_env_pkg::*;  
  
// Other imports and `includes  
`include spi_test_base.svh  
  
endpackage: spi_test_pkg
```